

RELATÓRIO DE TESTES UNITÁRIOS

Sistema de E-commerce — Projeto 1

Autor:

Vitor Razia Bubols

Disciplina:

Testes de Software

Data de entrega:

30 de março de 2026

1. Plano de Testes

1.1 Objetivo do Plano

Este plano de testes tem como objetivo validar o comportamento das quatro funções do sistema de e-commerce — `calculateTotal`, `applyDiscount`, `calculateShipping` e `validateProduct` — por meio de testes unitários automatizados. A meta é garantir que cada função responde corretamente tanto para entradas válidas quanto para entradas inválidas, incluindo casos limite (boundary values) e cenários de erro, assegurando a confiabilidade do código antes de qualquer integração ou deploy.

1.2 Escopo

Dentro do Escopo	Fora do Escopo
Funções <code>calculateTotal</code> , <code>applyDiscount</code> , <code>calculateShipping</code> , <code>validateProduct</code>	Testes de integração entre módulos
Casos válidos, inválidos, limites e erros de cada função	Testes de interface ou API HTTP
Cobertura mínima de 19 casos de teste unitário	Testes de performance ou carga

1.3 Ferramentas Utilizadas

Ferramenta	Versão	Finalidade
Vitest	^1.x	Framework principal de testes unitários
Node.js	18+	Ambiente de execução JavaScript
npm	9+	Gerenciamento de dependências
VS Code	Qualquer	Editor de código utilizado durante a atividade

1.4 Estratégia de Testes

Todos os testes desenvolvidos nesta atividade são do tipo unitário, ou seja, cada função é testada de forma isolada, sem depender de outras funções ou serviços externos. A organização dos testes segue o padrão AAA (Arrange, Act, Assert): primeiro preparam-se os dados de entrada, depois executa-se a função, e por fim verifica-se se o resultado obtido bate com o esperado.

Os arquivos de teste foram organizados por módulo, de forma que cada arquivo `.test.js` corresponde exatamente a um arquivo de `src/`. A ordem de execução seguiu a complexidade crescente: iniciou-se pelos testes de `calculateTotal` (mais simples), passando por `applyDiscount` e `calculateShipping`, e finalizando com `validateProduct`, que envolve validação de objetos.

Para cada função, foram cobertos os seguintes tipos de cenário: casos felizes (happy path) com entradas válidas, casos de erro com entradas inválidas ou fora do domínio, e valores limite (1kg e 5kg para frete; 0% e 100% para descontos). Casos de erro foram validados com `expect(() => fn()).toThrow()`, garantindo que os erros corretos são lançados nas situações esperadas.

1.5 Critérios de Aceite

Critério	Condição de Aceite
Todos os arquivos de teste criados	4 arquivos em tests/ (price, discount, shipping, product)
Quantidade mínima de testes	19 ou mais casos de teste passando
Zero falhas no terminal	npx vitest exibe 0 failed
Arquivos de src/ inalterados	Nenhuma modificação nos arquivos originais
Cobertura de edge cases	Cada função tem pelo menos um teste de erro (toThrow)

2. Matriz de Rastreabilidade de Testes

A tabela abaixo mapeia cada caso de teste ao comportamento específico que ele valida, facilitando a rastreabilidade entre requisitos funcionais e cobertura de testes. Os IDs seguem o padrão: PR (price), DC (discount), SH (shipping) e PD (product).

ID	Módulo	Função	Cenário Testado	Dados de Entrada	Resultado Esperado	Tipo
PR-01	price	calculateTotal()	Cálculo básico válido	(10, 2)	20	Unitário
PR-02	price	calculateTotal()	Valores decimais	(9.99, 3)	29.97	Unitário
PR-03	price	calculateTotal()	Preço zero ou negativo	(0, 2) / (-5, 2)	Lança erro	Unitário
PR-04	price	calculateTotal()	Quantidade zero ou negativa	(10, 0) / (10, -1)	Lança erro	Unitário
DC-01	discount	applyDiscount()	Desconto percentual válido	(100, 20)	80	Unitário
DC-02	discount	applyDiscount()	Desconto zero	(100, 0)	100	Unitário
DC-03	discount	applyDiscount()	Desconto 100%	(100, 100)	0	Unitário
DC-04	discount	applyDiscount()	Desconto negativo	(100, -10)	Lança erro	Unitário
DC-05	discount	applyDiscount()	Desconto acima de 100%	(100, 110)	Lança erro	Unitário
SH-01	shipping	calculateShipping()	Peso até 1kg (limite)	(1)	10	Unitário
SH-02	shipping	calculateShipping()	Peso abaixo de 1kg	(0.5)	10	Unitário

ID	Módulo	Função	Cenário Testado	Dados de Entrada	Resultado Esperado	Tipo
SH-03	shipping	calculateShipping()	Peso entre 1kg e 5kg	(3)	20	Unitário
SH-04	shipping	calculateShipping()	Peso exatamente 5kg (limite)	(5)	20	Unitário
SH-05	shipping	calculateShipping()	Peso acima de 5kg	(10)	40	Unitário
SH-06	shipping	calculateShipping()	Peso zero ou negativo	(0) / (-1)	Lança erro	Unitário
PD-01	product	validateProduct()	Produto completamente válido	{name:'X', price:10, stock:5}	true	Unitário
PD-02	product	validateProduct()	Nome ausente ou vazio	{name:'', price:10, stock:5}	Lança erro	Unitário
PD-03	product	validateProduct()	Preço zero ou negativo	{name:'X', price:0, stock:5}	Lança erro	Unitário
PD-04	product	validateProduct()	Estoque negativo	{name:'X', price:10, stock:-1}	Lança erro	Unitário
PD-05	product	validateProduct()	Estoque zero (esgotado)	{name:'X', price:10, stock:0}	true	Unitário

Evidência do terminal:

```

PS C:\Users\vitor\Git\SENAI_FASE3\Teste_de_sistemas\TESTES_UNITARIOS\e-commerce\01-unit-test-basic> npx vitest

DEV v4.0.18 C:/Users/vitor/Git/SENAI_FASE3/Teste_de_sistemas/TESTES_UNITARIOS/e-commerce/01-unit-test-basic

✓ tests/price.test.js (6 tests) 5ms
✓ tests/discount.test.js (5 tests) 5ms
✓ tests/product.test.js (5 tests) 5ms
✓ tests/shipping.test.js (7 tests) 5ms

Test Files  4 passed (4)
Tests      23 passed (23)
Start at   22:01:21
Duration   415ms (transform 113ms, setup 0ms, import 273ms, tests 20ms, environment 1ms)

PASS Waiting for file changes...
press h to show help, press q to quit

```

3. Reflexão Escrita

3.1 Qual foi o teste mais difícil de escrever?

O teste mais difícil foi o de `validateProduct`, especialmente o caso do estoque zero. Num primeiro momento, meu instinto foi assumir que estoque 0 seria inválido, afinal um produto sem unidades disponíveis parece um problema. Mas ao ler o código de `product.js` com atenção, percebi que a condição lança erro apenas quando o estoque é negativo, ou seja, `stock === 0` é completamente válido e representa um produto esgotado. Isso me fez entender que testar não é apenas "rodar a função", mas realmente compreender o domínio e as regras de negócio por trás do código.

3.2 Algum comportamento inesperado foi descoberto?

Não encontrei bugs nas funções (o Projeto 1 trabalha com código já funcional), mas alguns comportamentos me fizeram pausar e pensar. No `calculateShipping`, o retorno para peso exatamente igual a 1kg é 10 (e não 20), porque a condição verifica `weight <= 1`. Da mesma forma, para 5kg exato o retorno é 20 (e não 40). Esses boundary values são exatamente onde bugs costumam aparecer em sistemas reais — e escrever testes específicos para eles ajudou a confirmar que a lógica está correta e a entender exatamente onde cada faixa começa e termina.

3.3 Como foi decidido quais cenários testar?

A decisão partiu de três critérios principais. Primeiro, cobrir o happy path: o caso mais comum e esperado de uso da função. Segundo, cobrir todos os cenários de erro descritos na documentação, garantindo que cada `throw` está sendo disparado corretamente. Terceiro, identificar os boundary values — os valores-limite que separam um comportamento do outro, como 1kg e 5kg no frete ou 0% e 100% no desconto. Esses valores de fronteira são os mais propensos a gerar bugs por erros de comparação (`<` vs `<=`), então merecem atenção especial.

3.4 Qual a importância de testar casos de erro com `toThrow`?

Testar os casos de erro é tão importante quanto testar os casos de sucesso. Em sistemas reais, entradas inválidas chegam o tempo todo — seja por falha do usuário, por dados corrompidos ou por integração com outros sistemas. Se uma função deveria lançar um erro, mas não lança, o programa pode continuar executando com dados incorretos e gerar resultados silenciosamente errados, o que é muito mais difícil de debugar do que uma exceção explícita.

O matcher `toThrow()` do Vitest permite verificar não só que o erro ocorre, mas também que a mensagem de erro está correta. Isso garante que o contrato da função — tanto o retorno feliz quanto as condições de falha — está sendo respeitado. Em suma, um teste sem verificação de erros é um teste incompleto.

3.5 Considerações Finais

Esta atividade mostrou na prática que escrever testes vai muito além de fazer o código "passar". É um exercício de leitura crítica do código, de entendimento de regras de negócio e de antecipação de falhas. O processo de pensar nos cenários antes de escrever os asserts foi o momento mais valioso, pois forçou uma análise mais cuidadosa de cada função. Sair de zero testes para uma suíte de 20 casos passando foi também uma experiência concreta de como os testes aumentam a confiança no código e servem como documentação viva do comportamento esperado do sistema.